Introduction to ROS in SS25

Group Project: Autonomous Driving

Group 27

Keqing Zhang,Shuai Chen, Fangxu Han, Hao Zhang,Kaidong Guo

July 9, 2025

Project Description

This project aims to develop an autonomous driving system in a simulated urban environment using ROS and Unity. The vehicle must complete a predefined track as quickly as possible while staying on the road, avoiding collisions, and obeying traffic lights. Core tasks include building a perception pipeline with depth and semantic cameras, generating occupancy maps, implementing path and trajectory planners, handling dynamic traffic events (e.g., merging vehicles and emergency braking), and designing a control algorithm to drive the car.

Introduction to ROS in SS25 1
1 Introduction 1
1.1 Project Tasks and Responsibilities 1
1.2 Structure 1
2 Functionality and Performance 2
2.1 Perception Pipeline 2
2.2 Point Cloud Generation 2
2.3 Octomap Generation 3
2.2 RVIZ Visualization 4
3 Traffic Light Detection (YOLOv5) 7
4 Planning 9
5 State Machine 14
6 Controller 20
7 Results and Evaluation 23
8 Challenges 23
9 Conclusion 24
10 References 24

1 Introduction

This report documents the implementation and results of our ROS1-based autonomous driving project using Unity simulation and RVIZ, focusing on completing a predefined urban track efficiently while adhering to traffic rules, detecting traffic lights, and avoiding obstacles in real time. The perception pipeline leverages depth cameras to generate point clouds, which are transformed into Octomap and projected into 2D costmaps to support planning and obstacle avoidance. For traffic light detection, YOLOv5 is utilized instead of the semantic camera, enabling flexible dataset training, lightweight deployment, and higher control over detection quality under various lighting conditions.

1.1 Project Tasks and Responsibilities

Task	Contributor	Packages/Tools Utilised
Point Cloud Generation & Octomap Mapping	Hao Zhang Kaidong Guo	depth_image_proc, octomap_ros, octomap_server, RVIZ
Traffic Light Detection	Hao Zhang	YOLOv5, RGB Camera, depth camera, PyTorch
State Machine Implementation	Keqing Zhang	SMACH
Global Planner	Shuai Chen	move_base, nav_msgs, tf
Local Planner	Shuai Chen	teb_local_planner
Controller Design	Fangxu Han	Ackermann Model, Proportional Controller
System Integration & Testing	All	Unity Simulation, RVIZ, ROS1

1.2 Structure

The project is modularized as follows:

- Perception pipeline: Depth camera data processing, point cloud generation, Octomap and costmap creation, YOLOv5-based traffic light detection.
- Planning: Global and local planning using `***move_base***` and `***teb_local_planner***`.
- State machine: Manages mission logic and traffic light compliance.
- Controller: Ackermann steering with PID/Proportional Cascade for tracking planned trajectories.

- RVIZ visualization: Real-time monitoring of point clouds, Octomap, costmaps, planned paths, and system status.

In the following figure 1, the complete node, topic structure graph is shown



Fig1.1. rqt graph of the whole

The rqt graph above illustrates the overall ROS communication architecture of the autonomous driving system. It visualizes the nodes and topics involved in perception, planning, decision-making, and control. Sensor data from devices such as RGB and depth cameras are processed through dedicated nodes, including YOLOv5-based detection modules. The ***/traffic_light_info_node*** integrates RGB and depth data to extract traffic light status and distance, publishing this information to the ***/traffic_light_info*** topic. The state machine node subscribes to this topic, along with additional inputs such as turning signals and lane change detection, and determines the current driving mode. Based on the selected mode, control commands are published via ***/ackermann_cmd*** and ultimately sent to the vehicle through ***/car_cmd***. This graph effectively demonstrates the modular and hierarchical structure of the system.

2 Functionality and Performance

2.1 Perception Pipeline

Using ***depth_image_proc*** depth images are converted into 3D point clouds to represent the environment, which are then fed into ***octomap_ros*** and ***octomap_server*** to generate Octomap for 3D occupancy mapping. The Octomap is further projected into 2D costmaps to provide the planner with real-time obstacle and free space information. Kalman filtering is optionally applied for sensor fusion if IMU integration is used for smoother odometry, and RVIZ is used for visualization during testing to verify correctness of point cloud and Octomap generation.

2.2 Point Cloud Generation

To enable environmental perception in our ROS-based autonomous driving project, we utilized the ***depth_image_proc*** package to convert raw depth images into **3D point clouds** for mapping and navigation.

In our system, the Unity simulation environment publishes raw depth images on: ***/Unity_ROS_message_RX/OurCar/Sensors/DepthCamera/image_raw*** and the camera calibration parameters on:
/Unity_ROS_message_RX/OurCar/Sensors/DepthCamera/camera_info

The ***depth_image_proc*** node subscribes to these topics and processes the depth data using the intrinsic parameters to generate a ***sensor_msgs/PointCloud2*** message, which we configured to publish on: ***/points***

This topic is then utilized by mapping and visualization modules. In RViz, we visualized the **point clouds as dense, colored scatter plots**, allowing us to intuitively check the completeness and accuracy of environmental perception during the vehicle's movement in simulation.

We monitored the topic using: ***rostopic echo /points*** and checked the connection graph with ***rqt_graph*** to confirm the data flow:

DepthCamera (***image_raw***, ***camera_info***)

→ ***depth_image_proc***

→ ***/points***

This pipeline ensured that our system could perceive and represent 3D structures in real time, providing the necessary input for OctoMap and planning modules.

2.3 Octomap Generation

After generating point clouds, we used ***OctoMap*** to build a probabilistic 3D occupancy map of the environment. OctoMap maintains an octree structure, where each voxel stores the occupancy probability of that space, allowing for efficient storage and updating during runtime.

The ***OctoMap server*** subscribes to the ***/points*** topic, processes the incoming point cloud data, and publishes the generated maps on:

/octomap_full
/octomap_binary

/projected_map
/octomap_point_cloud_centers

For visualization, we primarily used ***/octomap_full*** and ***/projected_map*** in RViz. These topics allowed us to see voxel-based maps of the environment, showing occupied areas as colored cubes while leaving free and unknown areas transparent. This dynamic visualization helped us confirm whether mapping was proceeding correctly as the vehicle moved and rotated.

The TF tree, published under ***/tf***, ensured consistent alignment between the world frame, the vehicle's base frame (OurCar/CoM), and the camera frame, which is critical for correct mapping.

By using ***rostopic echo /octomap_full*** and ***rostopic echo /projected_map***, we could verify that the OctoMap node was publishing data in real time, while rostopic list confirmed that all expected topics were active during operation, including:

/Unity_ROS_message_RX/OurCar/CoM/pose
/Unity_ROS_message_RX/OurCar/CoM/twist
/move_base_simple/goal

Through this process, we learned how point cloud data is transformed into structured occupancy maps, and how these maps are used in planning and navigation. We also learned the importance of tuning voxel resolution for balancing detail with computational efficiency, which is a common trade-off in robotics applications.

2.2 RVIZ Visualization

RVIZ is used to visualize point clouds, Octomap, costmaps, planned trajectories, and vehicle status in real time, providing an effective debugging and presentation tool during testing and final demonstrations.

In the environment perception module, we first use the ***depth_image_proc*** package to process depth images and generate dense point cloud data, which represents the spatial structure surrounding the vehicle. Then, the ***octomap_server*** is used to convert the point cloud into a voxel-based map (Octomap), enabling sparse 3D mapping and efficient environment representation. This voxel map not only serves for visualization but also provides essential support for path planning and obstacle avoidance.



Fig2.1. Unity map

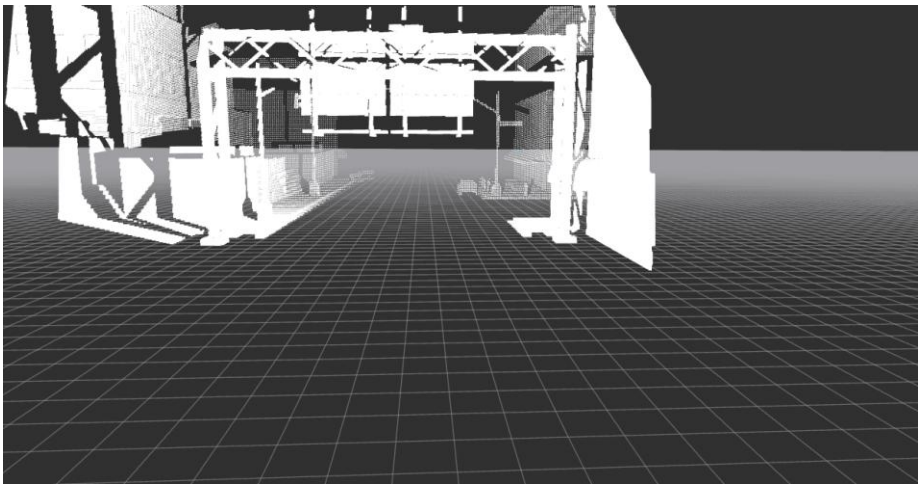


Fig2.2. Point cloud

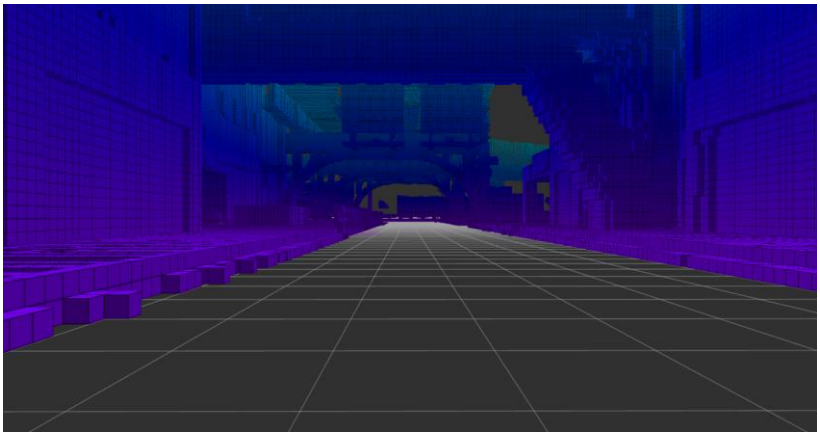


Fig2.3. voxel-based map

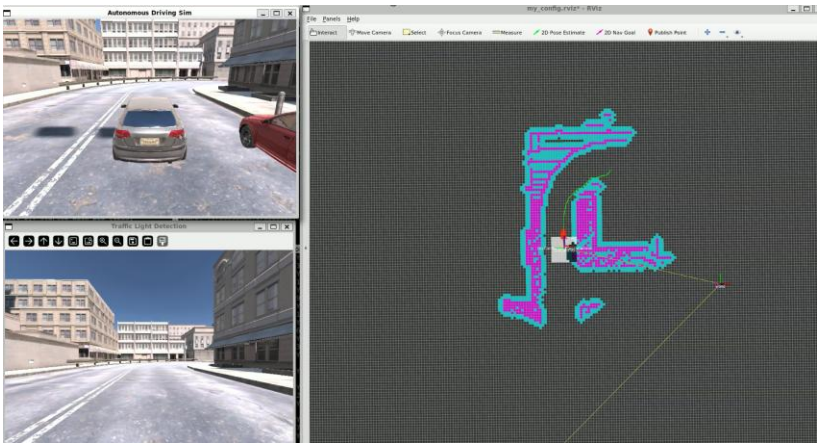


Fig2.4. Costmap

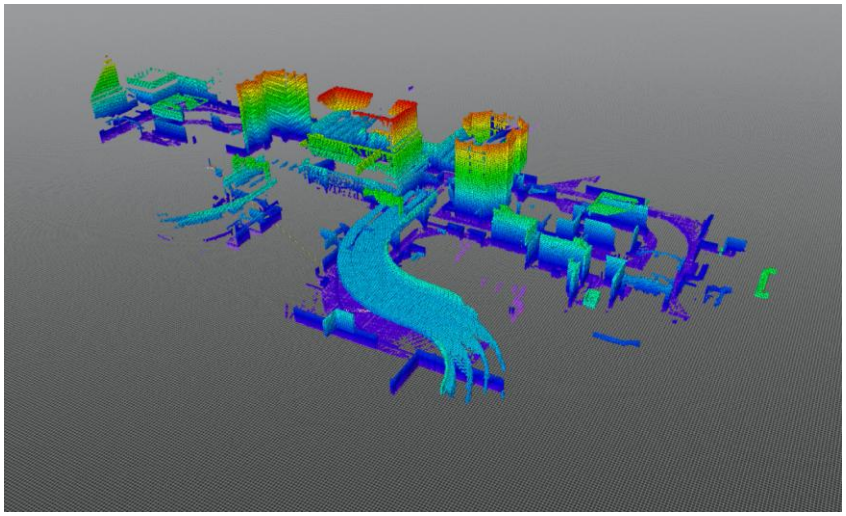


Fig2.5. Whole Voxel map of the Simulation

3 Traffic Light Detection (YOLOv5)

Traffic lights are a crucial part of urban roads, making their recognition essential in autonomous driving. In this project, we implemented both traffic light color and distance detection without using a semantic camera.

We used an RGB camera to receive image data, which was processed by YOLOv5 to detect the "traffic light" class. Then, OpenCV was used to identify the traffic light color: red, yellow, or green. To improve detection stability, we implemented a voting mechanism. Specifically, a colour change is confirmed only when the same color is detected in three consecutive high-frequency frames. For example, if the current state is red and the next three detections are "red-green-red", the output remains red. Only when three consecutive detections are "green-green-green" will the output switch to green.

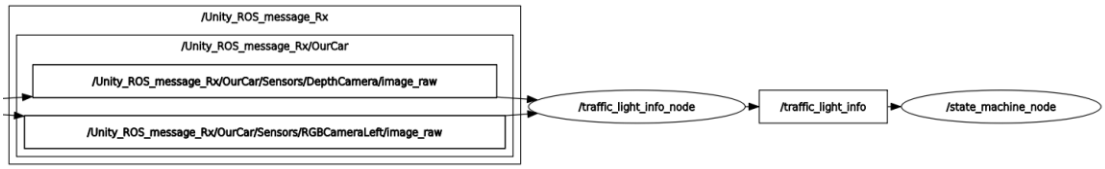


Fig3.1. rqt graph of traffic light detction

As shown in the rqt graph above, the node ***/traffic_light_info_node*** receives data from both the RGB camera and the depth camera. After processing with YOLOv5 and OpenCV, it publishes the topic ***/traffic_light_info***. This topic contains two key pieces of information: ***/traffic_light_status*** and ***/traffic_light_distance*** the detected traffic light color and the distance between the vehicle and the traffic light. The state machine subscribes to this topic and determines whether to decelerate based on the color and the distance data.

Considering that in real driving scenarios, deceleration is typically triggered near the stop line before the crosswalk, we used a depth camera to obtain depth information. By calibrating this depth data with the image data, we were able to detect the real-time distance between the vehicle and the traffic light.

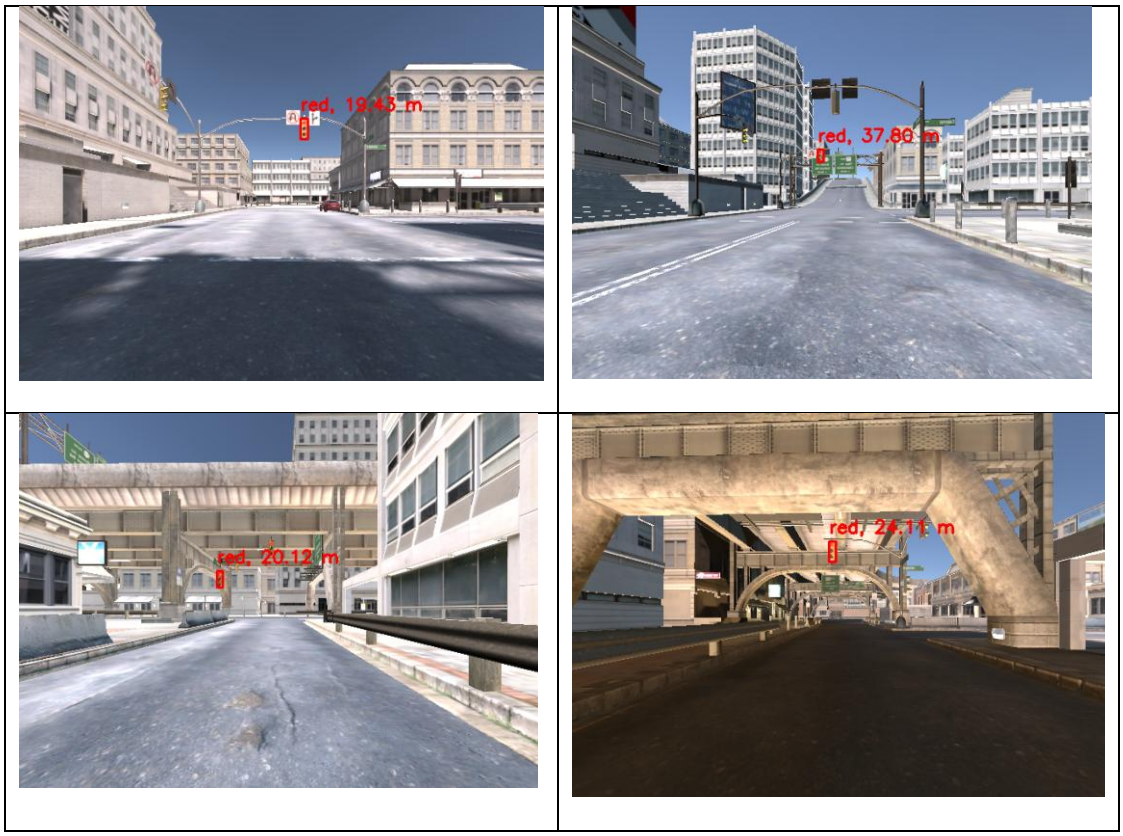


Fig3.2. Traffic light detection in unity

4 Planning

ROS officially provided a navigation package ***move_base***, which is a powerful tool to handle autonomous navigation tasks. It plays a central role in our planning pipeline by enabling the car to reach the target position through path planning and obstacle avoidance.

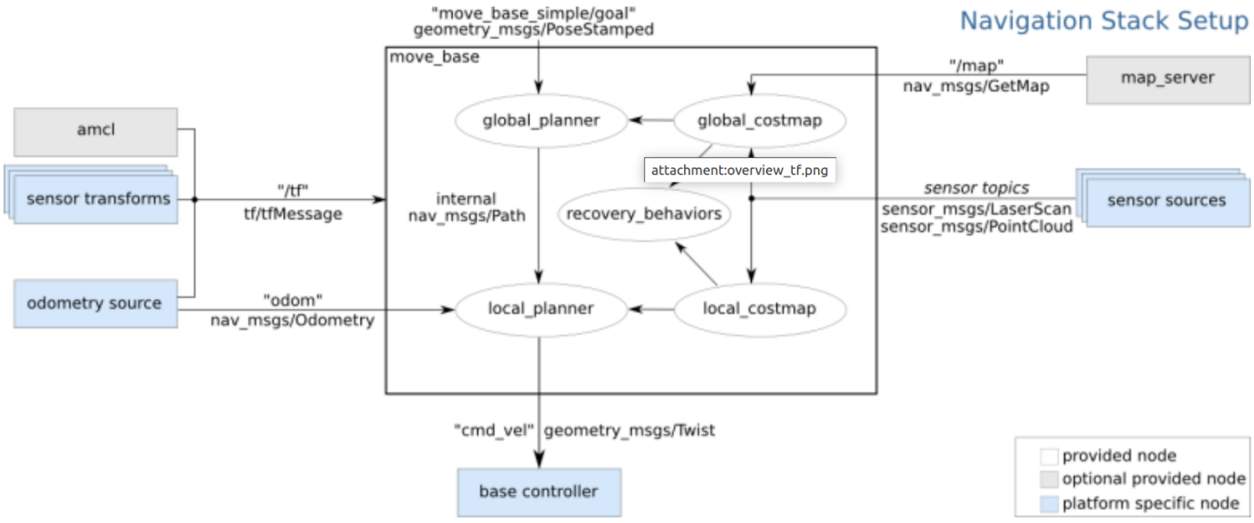


Fig4.1. Move base node

In ROS, ***move_base*** is responsible for the following:

- **Localization**: Allows the robot to be aware of its position within the environment. By specifying the relevant interface in the configuration files, the planners can acquire the robot's current pose.
- **Path Planning**: Finds a feasible path between the starting point and the target. By subscribing to the pose information published from Unity, the robot receives trajectory points within the Unity environment. A series of these points are sent to the global planner to generate a global path, which is then passed to the local planner in the format of ***nav_msgs/Path***.
- **Obstacle Avoidance:** Identifies and avoids obstacles based on the costmap generated by the perception module. By configuring the per-generated point cloud data as the sensor source and setting the map information to ***/projected_map*** in RViz, obstacle avoidance can be achieved.

- **Recovery Behaviors:** Executes corrective actions when the robot encounters obstacles or fails to find a path. The ***clear_costmap*** plugin is used to clear costmap data within a certain range, and the ***move_slow_and_clear*** plugin enables the robot to slowly move a short distance to bypass obstacles.

Configuration Files:

In this project, six main parameter configuration files are located under the path ***planning/local_planner/params/***. These files define the settings for the costmap system, local planner, and the main navigation node:

- ***costmap_common_params.yaml***
  Defines parameters shared by both the global and local costmaps, such as plugin loading order and the robot's footprint model.
- ***global_costmap_params.yaml***
  Specifies the configuration of the global costmap, including global reference frame, static map layers, and plugin setup. It is primarily responsible for long-range path planning.
- ***local_costmap_params.yaml***
  Configures the local costmap for real-time obstacle avoidance. Key settings include rolling window behavior and the obstacle sensing range.
- ***static_map_params.yaml***
  Sets the basic parameters for the static map layer, such as the subscribed map topic and the map update behavior.
- ***move_base_params.yaml***
  Provides core parameters for the move_base node, including planner/controller execution frequency, goal position/orientation tolerances, and recovery behavior settings.
- ***teb_local_planner_params.yaml***
  Configures the TEB local planner with motion constraints, velocity limits, obstacle avoidance radius, and trajectory optimization weights.

4.1 Global Planner

move_base does not support path planning with custom-defined way-points. According to the task requirements, we manually control the vehicle using the keyboard and record its trajectory by listening to the pose information published from Unity. The ***multi_goal_manager*** package then publishes this series of trajectory points to the global planner to generate the global path.

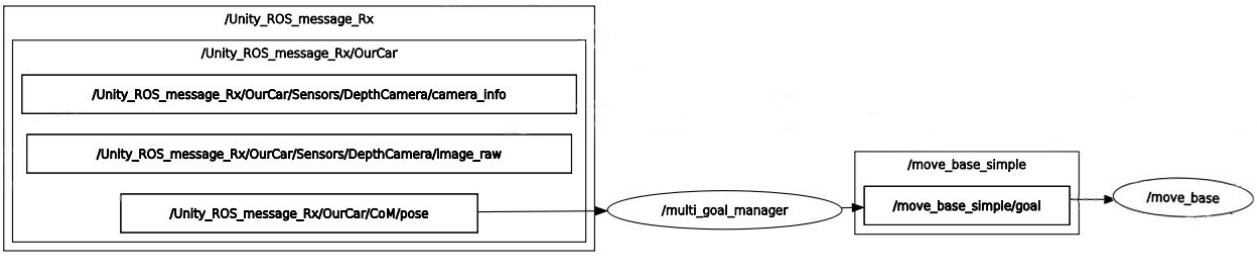


Fig4.2. rqt grapgh of global planner

- **Package *multi_goal_manager***:

This package implements a multi-goal navigation node in ROS, allowing the robot to sequentially navigate through a series of waypoints. After reaching each goal, the robot automatically proceeds to the next until all goals are completed. Its main functionalities are as follows:

  - Subscribe to current position:
    The node subscribes to the ***/Unity_ROS_message_Rx/OurCar/CoM/pose*** topic to receive the current position of the vehicle.
  - Define and publish goals:
    The predefined goal points are constructed as ***PoseStamped*** messages and published to the topic ***/move_base_simple/goal***.
  - Check goal arrival:
    Upon receiving the vehicle's current position, the node calculates the distance to the current goal. If the distance is less than a threshold of 4 meters, the goal is considered reached.
  - Publish the next goal:
    Once the current goal is reached, the next goal in the list is published. This process continues until all goals have been sent.

- **Dijkstra Algorithm:**

The commonly used global planning algorithms in **move_base** include **A*** and **Dijkstra**. In this project, we chose Dijkstra as the global planner mainly because it performs more robustly in scenarios where the map information is incomplete.

By setting the parameter ***base_global_planner*** to ***global_planner/GlobalPlanner*** in the ***move_base.params.yaml*** file and enabling the ***use_dijkstra*** option, the Dijkstra algorithm is activated.

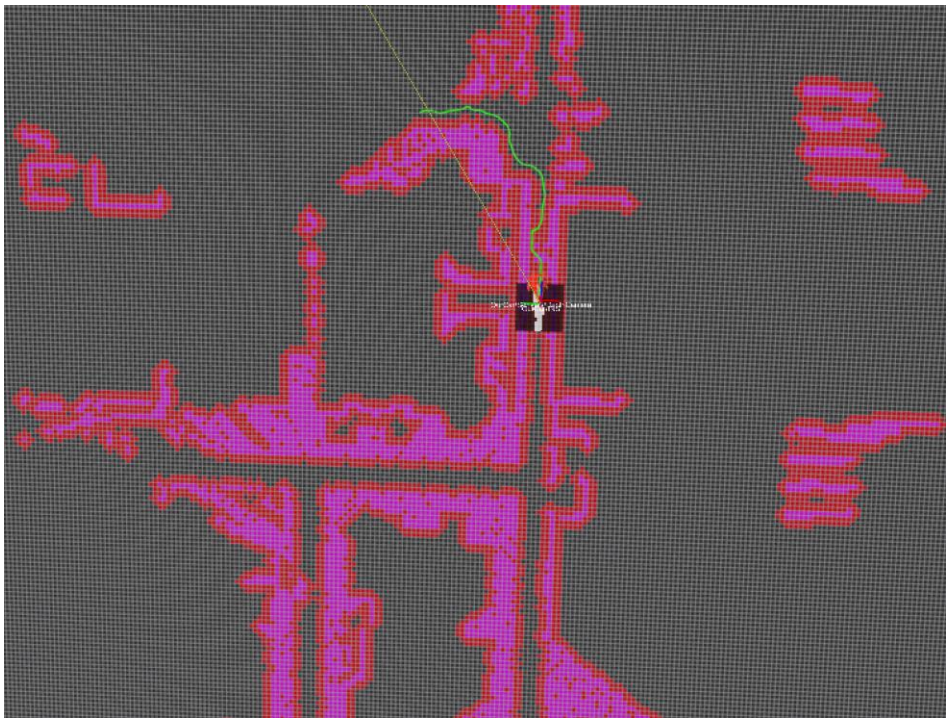


Fig4.3. Global planner in Costmap

4.2 Local Planner

In this project, we utilize the widely used ROS package ***teb_local_planner*** as the local path planner and package ***odom_from_unity*** to converts the massage.

- Package ***teb_local_planner***:

It is integrated within the ***move_base*** framework. It receives the global path (***nav_msgs/Path***), the current robot pose from ***/odom***, and local **costmap** data from ***/local_costmap***, then outputs motion commands to ***/cmd_vel***. This enables the robot to dynamically avoid obstacles and reach the target point reliably even in complex environments.

- package ***odom_from_unity***:

  To obtain the required odometry (odom) data for the planning pipeline, we use the package ***odom_from_unity***, which converts the vehicle's pose (Pose) and velocity (Twist) information published by Unity into the standard ROS message format ***nav_msgs/Odometry***. This ensures compatibility with modules such as ***move_base*** that rely on standard odometry inputs.

Based on the Timed Elastic Band (TEB) algorithm, this planner is capable of real-time trajectory optimization that respects dynamic constraints, ensures obstacle avoidance, and maintains trajectory smoothness. Compared to traditional methods like DWA, TEB generates more natural and controllable trajectories by incorporating velocity, acceleration, and turning limitations, which greatly reduces the accuracy demands on the controller.

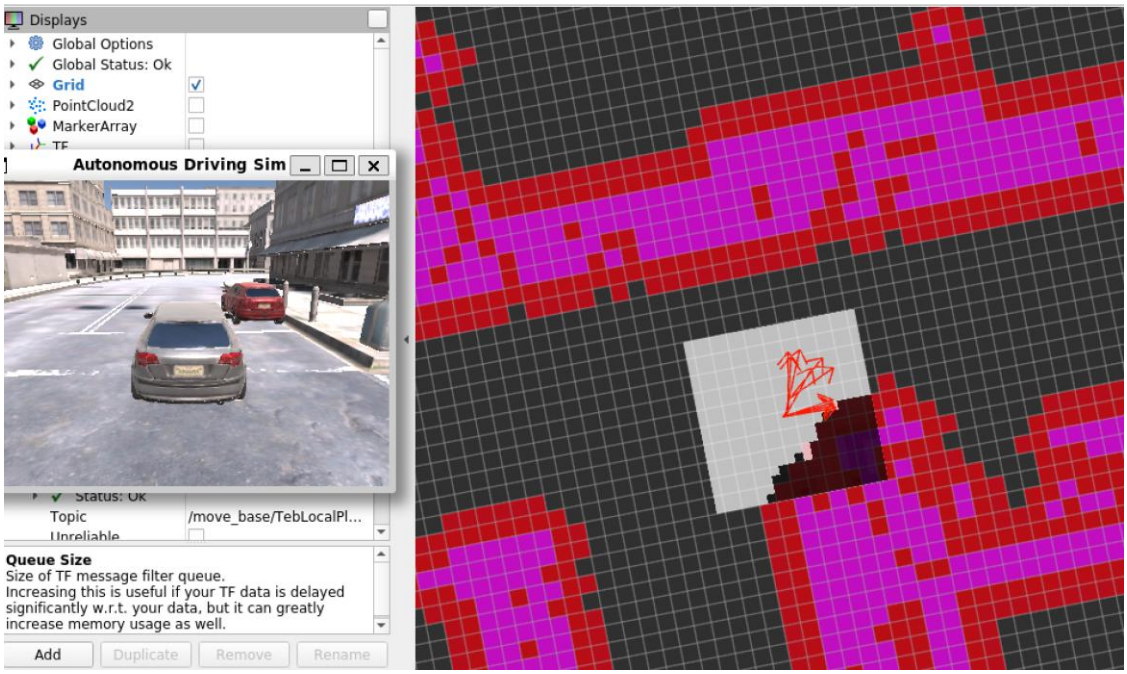


Fig4.4.1. Vector of local planner in Costmap

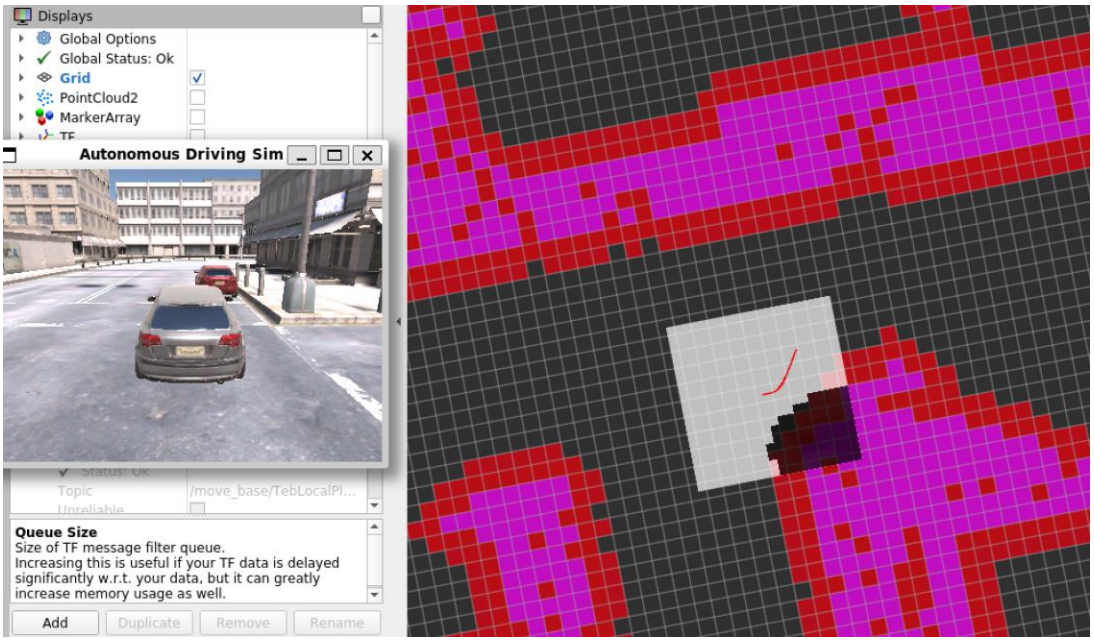


Fig4.4.2. Path of local planner in Costmap

5 State Machine

This module serves as a decision-making unit that subscribes to perception data (traffic lights, vehicle pose, merging detection, and velocity) and publishes driving commands ***/controller_command*** to control vehicle behavior. The core functionality is implemented using the ***smach*** library to create a **finite state machine (FSM)** that transitions based on the environment and driving conditions.

5.1 Introduction to SMACH

SMACH is a task-level architecture for rapidly creating complex robot behavior using hierarchical and concurrent state machines. It is a Python-based library designed for flexibility, modularity, and integration with the ROS ecosystem.

Originally developed to manage robot decision-making in dynamic environments, SMACH allows you to structure behavior as a set of states, each handling specific tasks and transitioning based on logic or sensor input. It supports the nesting of state machines (hierarchical) and concurrent execution of multiple behaviors.

SMACH also comes with tools like ***smach_viewer***, which provides graphical visualization of state transitions in real time—useful for debugging and understanding robot logic.

5.2 Code Structure Overview

This module is implemented as a single ROS node using Python and the smach library. It is logically organized into the following sections:

- Shared Data Class (TrafficLightData)
  - Centralized container for all perception data (traffic light info, pose, turning, merging status).
  - Shared across all states through global access.
- Callback Functions

  Handle incoming sensor data via ROS subscribers:
  - ***/traffic_light_info*** → updates traffic light color and distance.
  - ***/cmd_vel*** → used for turning detection.
  - ***/amcl_pose*** → monitors position to detect return to origin.
  - ***/is_merging*** → indicates merging situations.
- State Classes

  Each subclass of smach.State handles one driving behavior:
  - NormalDriving
  - EmergencyBrake
  - Turning
  - VehicleMerge
  - ReachGoal
- Each state:
  - Processes relevant conditions from the shared data,
  - Publishes a command to ***/controller_command***,
  - Returns a transition outcome string.
- State Machine Assembly
  - All states are added to the ***smach.StateMachine*** objects with clearly defined transitions.
  - Entry point is ***NormalDriving***; final state is ***ReachGoal.***

- Execution Setup

- ROS node initialization.
- State machine construction and execution.
- Launches optional ***smach_viewer*** for visualization.

5.3 Scenarios and State Logic

- Red light within 15m → Emergency Brake.
- Turning detected (based on angular velocity) → Enter Turning state.
- Merging situation detected → Enter Vehicle Merge state.
- Returned to start position after driving → Reach Goal (stop).
- Otherwise → Continue Normal Driving.

State	Trigger Condition	Transition Target
NormalDriving	Turning detected (angular.z > threshold)	Turning
	Merging detected (/is_merging = True)	VehicleMerge
	Red light within *15*m	EmergencyBrake
	Returned to origin after movement	ReachGoal
	None of the above	NormalDriving (loop)
EmergencyBrake	Traffic light turns green or is far (>20m)	NormalDriving
Turning	Turning ends	NormalDriving
VehicleMerge	Merging ends	NormalDriving
ReachGoal	Terminal state, vehicle stopped	Remains in ReachGoal

5.4 Input and Output Interfaces

Inputs:

Topic	Description
/traffic_light_info	Custom message (TrafficLightInfo) containing traffic light color (color) and distance (distance) to traffic light.
/is_merging	Boolean signal indicating whether the vehicle is merging into another lane.
/cmd_vel	Angular velocity (angular.z) used to detect whether the vehicle is turning.

Output:

Topic	Message Type	Description
/controller_command	Int32	0 = emergency stop& reach the goal
		1 = Drive Normally
		2 = Turning
		3 = Merging

5.5 State Behavior and Actions

- NormalDriving
  - Default driving state.
  - Publishes 1 to /controller_command.
  - Monitors for merging, turning, red lights, or goal reached.
- EmergencyBrake
  - Active when a red light is detected within 20 meters.
  - Publishes 0 to stop.
  - Transitions back when safe.

- Turning
  - Active if turning behavior is detected (angular.z above threshold).
  - Publishes 2 to indicate turning.
  - Resumes to driving when angular velocity drops.
- VehicleMerge
  - Active during merging behavior (/is_merging = True).
  - Publishes 3.
  - Loops until merging ends.
- ReachGoal
  - Final state when vehicle returns to (0,0) after starting.
  - Publishes 0 and terminates further driving logic.

5.6 State Machine Diagram

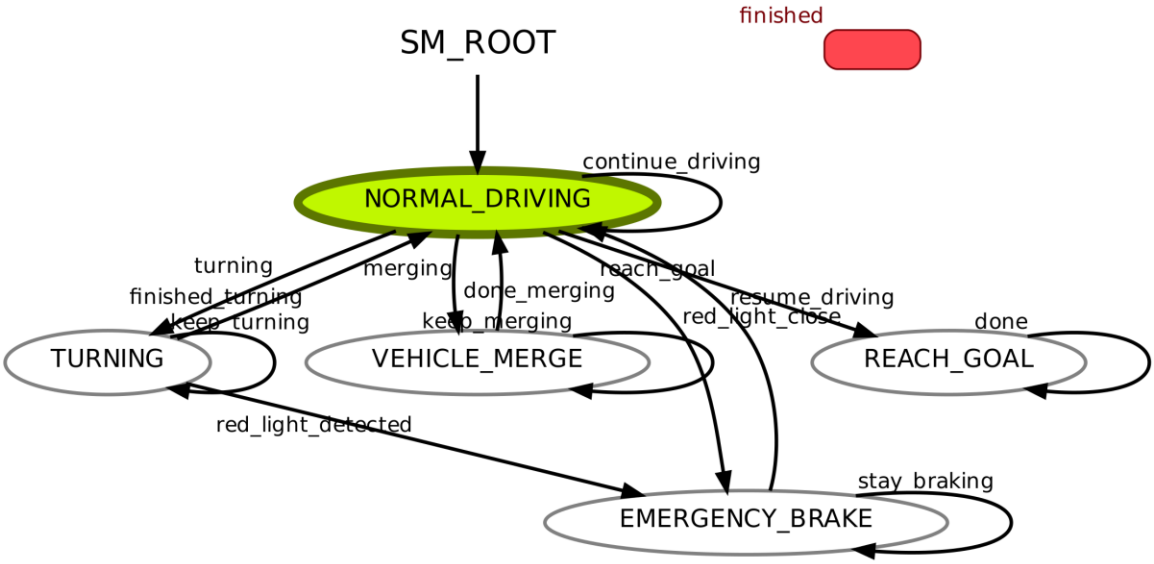


Fig5.1. State Machine Diagram

5.7 Experimental Notes

- Turning detection uses angular velocity threshold and duration check.
- Position checking ensures the vehicle has moved before checking for goal return.
- All states periodically sleep to allow updates from subscribers.

5.8 Installation and Execution

1) Before running the state machine, make sure the required SMACH packages are installed:

   sudo apt install ros-noetic-smach

   sudo apt install ros-noetic-smach-ros

   sudo apt install ros-noetic-smach-viewer

2) Once the packages are installed and your workspace is built:

   roslaunch statemachine_pkg state_machine.launch

3) You can launch the SMACH viewer in a separate terminal to monitor state transitions in real time:

   rosrun smach_viewer smach_viewer.py

6 Controller

In this section, the ***Ackermann steering model*** is used to describe the vehicle's turning dynamics. The controller receives real-time vehicle state information from ***/odom,*** target velocity commands from ***/cmd_vel*** and driving state signals—such as traffic lights or lane-change events—from the perception module via state machine to ***/controller_command***. Based on these inputs, the controller computes throttle, brake, and steering outputs through a **Proportional-Cascade**, which are published to ***/car_command*** to drive the vehicle accordingly.

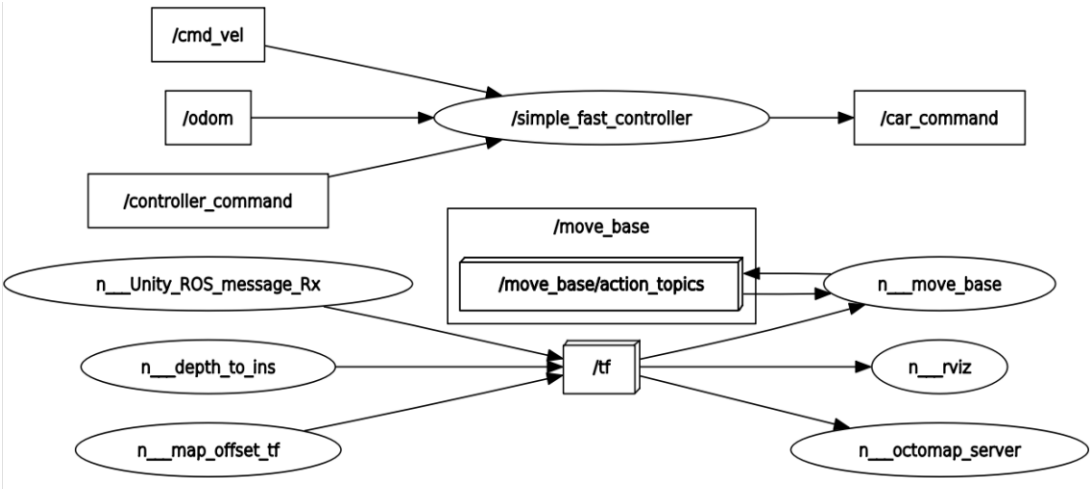


Fig6.1. rqt graph of closed loop controller

A simpler ***open-loop*** control strategy is also introduced, which relies solely on ***/cmd_vel*** as input. This approach can yield better performance under the simplified conditions of the current simulation scenario.

6.1 Ackermann Steering Model

In this project, the vehicle's steering system is modeled using the **Ackermann steering geometry**, which is widely used for road vehicles to accurately capture the relationship between the steering angle and the vehicle's turning curvature. The Ackermann model assumes that the front wheels turn while the rear wheels follow a fixed track, enabling the vehicle to follow a circular path without lateral tire slip.

In our implementation, the desired steering angle δ is computed based on the commanded forward velocity v_x and yaw rate ω_z using the following kinematic relation:

$$\delta = \arctan\left(\frac{b \cdot \omega_z}{v_x}\right)$$

where b is the vehicle's wheelbase.

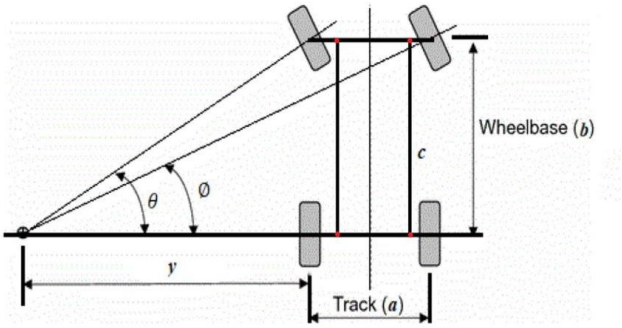


Fig6.2. Ackermann Steering Model

6.2 Closed-loop Control

Based on the previously introduced Ackermann steering model, a **Proportional (P) controller** is employed to enhance the system's robustness and control accuracy under real-world driving conditions. Specifically, the P controller is used to determine the vehicle's longitudinal acceleration, as shown in Equation

$$a_{x|ctrl} = k_p(v_{desired} - v_{real})$$

with $v_{desired}$ derived from node ***local_planner*** and v_{real} from node ***odom_from_unity***

A full PID controller is not adopted in this case because the current driving scenario is relatively simple. Introducing integral and derivative terms would unnecessarily complicate the system and could even degrade control accuracy. After parameter identification, a proportional gain of P = 1 was selected as the most effective setting for this environment.

6.3 Open-loop Control

To achieve better control performance in the current idealized simulation environment, the system can be simplified by removing odometry feedback from ***/odom*** and instead using the continuously updated desired values from the local planner to directly control the vehicle, as illustrated in **Fig. 6.3.** This **open-loop control** approach offers several advantages in our simulation project:

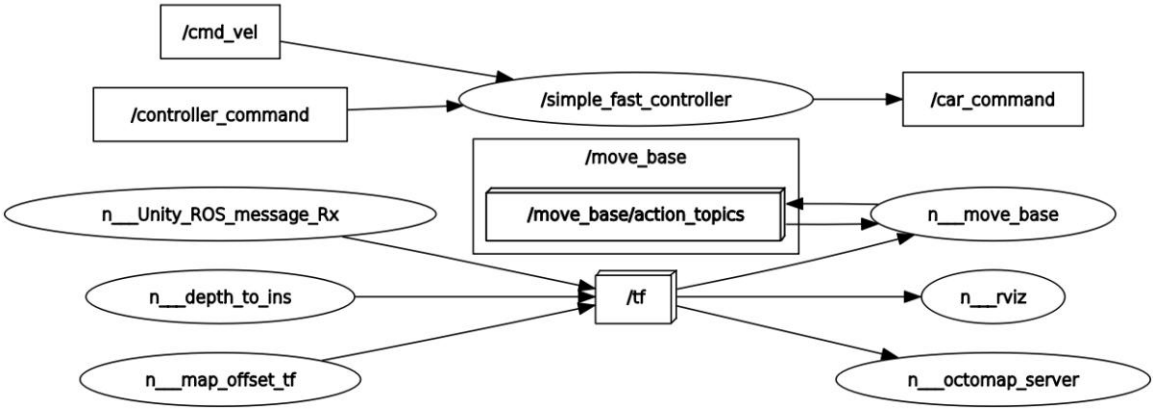


Fig6.3. Rqt graph of open loop controller

1) **Reduced System Complexity**: By eliminating the need for odometry feedback, we significantly simplify the communication interfaces and avoid introducing unnecessary noise or oscillations into the control loop.
2) **Improved Stability**: The simulator has been validated to provide highly accurate motion execution. In this context, simple P control may cause throttle fluctuations at high update rates, negatively affecting vehicle smoothness. In contrast, open-loop control yields stable and predictable outputs.
3) **Greater Robustness**: Odometry data from ***/odom*** may become unreliable due to communication delays, occasionally causing the controller to fail. Open-loop control avoids this risk and has proven to be more reliable in simulation. In fact, experimental results show that open-loop control enables faster vehicle operation and reduces the total route completion time by at least 40%.

Although open-loop control performs well in this experiment, it also exhibits certain limitations. For instance, in real-world scenarios involving terrain slope changes, the inability to adjust speed automatically may lead to overshooting or insufficient driving force. Additionally, model inaccuracies can cause errors to accumulate over time, especially in the absence of feedback correction. Furthermore, open-loop control lacks

the ability to effectively eliminate disturbances caused by environmental noise commonly encountered on real roads.

6.4 Future Work

According to several studies on autonomous driving systems used by major automotive manufacturers, **Model Predictive Control (MPC)** has emerged as the mainstream control strategy worldwide. MPC offers several advantages that make it well-suited for real-world driving scenarios: it can handle complex road geometries, dynamically changing environments, and system constraints while optimizing future control actions over a prediction horizon. Compared to simpler control methods, MPC provides greater precision, stability, and adaptability particularly in the presence of uncertainties, varying road conditions, and multi-objective tasks such as lane keeping, obstacle avoidance, and speed regulation. As autonomous vehicles move toward higher levels of autonomy, MPC is increasingly favored as a robust and flexible framework for both lateral and longitudinal vehicle control.

7 Results and Evaluation

- Visualized Octomap, point clouds, and planned paths in RVIZ, confirming environment representation and path tracking.
- Accurate detection of traffic lights using YOLOv5 with effective stop/go transitions.
- Implemented a state machine that recognized traffic light states and driving modes (e.g., turning or lane-following) and communicated decisions to the controller.
- Created new custom message types (e.g., Int32 output from the state machine) to interface cleanly with the controller logic.
- Successfully followed the predefined track while avoiding obstacles and adhering to traffic lights.
- Achieved stable mission completion with consistent control using the proportional Ackermann controller.

8 Challenges

- Parameter tuning for ***teb_local_planner*** and controller for stability and responsiveness.
- Fine-tune TEB parameters to improve local planning smoothness
- Integrate PID/MPC-based controller for smoother actuation

- The current algorithm struggles to accurately recognize each traffic light in complex scenarios, such as when multiple lights overlap or are partially occluded, and therefore requires optimization.
- YOLOv5 detection required augmentation for robust detection under simulation lighting.
- Ensuring alignment between RGB detection bounding boxes and depth images for distance measurement.

9 Conclusion

The autonomous driving project under ROS1 was successfully implemented with clear modularity, demonstrating perception, planning, control, and visualization workflows in Unity simulation. Replacing the semantic camera with YOLOv5 provided flexibility and adaptability for real-world and simulation scenarios while maintaining accurate detection and system stability.

10 References

[1] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, “OctoMap: An efficient probabilistic 3D mapping framework based on octrees,” Autonomous Robots, 2013. Software available at http://octomap.github.com.

[2] A. Hornung et al., 'OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees', Autonomous Robots, 2013

[3] J. Levinson et al., "Towards fully autonomous driving: Systems and algorithms," 2011 IEEE Intelligent Vehicles Symposium (IV), Baden-Baden, Germany, 2011, pp. 163-168, doi: 10.1109/IVS.2011.5940562. keywords: {Trajectory;Vehicles;Calibration;Laser beams;Vehicle dynamics;Planning;Software},

[4] C. Rösmann,W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, “Efficient trajectory optimization using a sparse model,” in 2013 European Conference on Mobile Robots, pp. 138–143, 2013.18

[5] E. Yurtsever, J. Lambert, A. Carballo and K. Takeda, "A Survey of Autonomous Driving: Common Practices and Emerging Technologies," in IEEE Access, vol. 8, pp. 58443-58469, 2020, doi: 10.1109/ACCESS.2020.2983149. keywords: {Automation;Task analysis;Systems architecture;Accidents;Planning;Vehicle dynamics;Robot sensing systems;Autonomous vehicles;control;robotics;automation;intelligent vehicles;intelligent transportation systems}

[6] D.-S. Tran, N.-H. Ho, H.-J. Yang, E.-T. Baek, S.-H. Kim, and G. Lee, “Real-time hand gesture spotting and recognition using rgb-d camera and 3d convolutional neural network,” Applied Sciences, vol. 10, no. 2, 2020.